Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

laboratory work no. 3

Course: Operating Systems
Topic: The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. Biriucov Alexandr
Checked by: Reițman Patricia

Chișinău, 2026

Purpose of the work

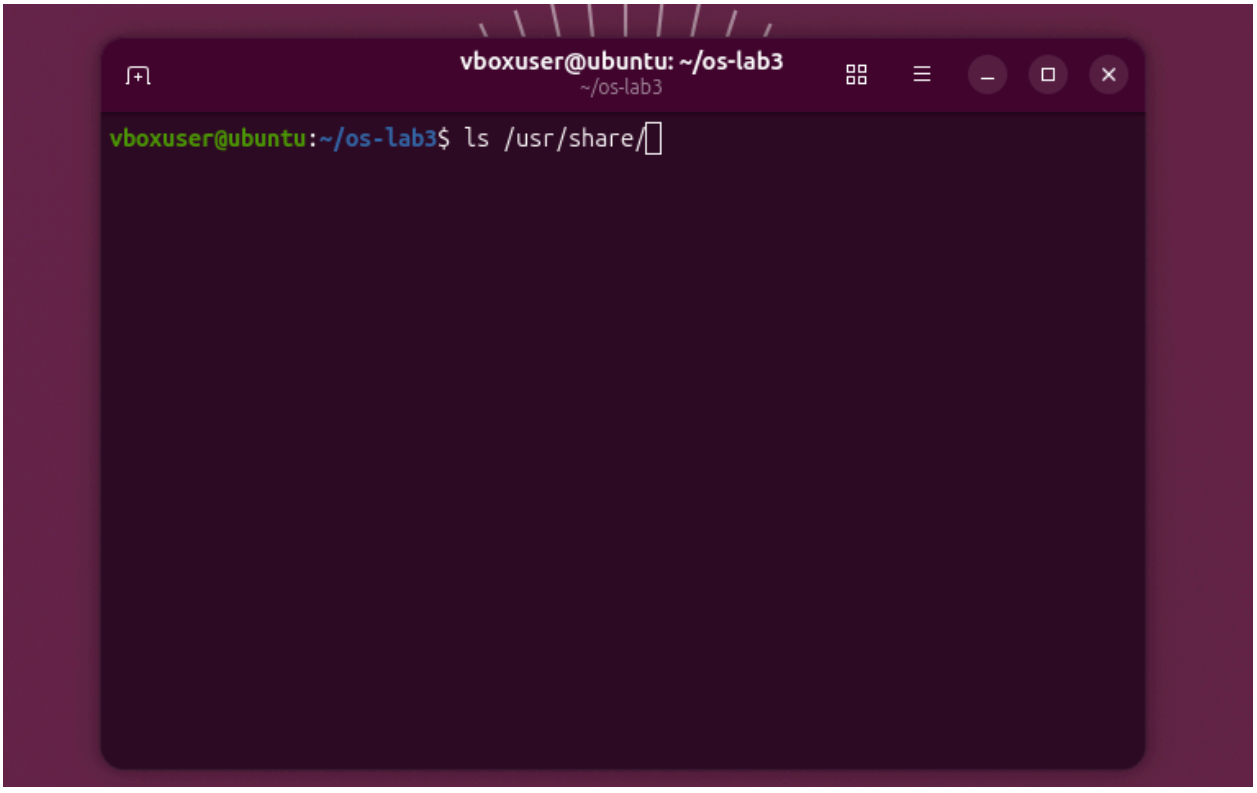
The purpose of this work is to learn to work fast and confidently in the Linux bash shell: to use tab completion, history and built-in help; to predict how the shell expands a command line (globbing, quoting, variable and command substitution) before the command runs; to redirect standard input, output and error streams and chain commands with ;, && and ||; to answer questions about data with one-line pipelines of small text filters (grep, cut, sort, uniq, wc, head, tail, tr); and to turn working one-liners into a first shell script.

Part 1. Moving fast and finding help

1.1 Tab completion and history

```
ls /usr/sh<Tab><Tab>
cd /et<Tab>
```

Observation: with /usr/sh there is only one match, so Tab completed it to /usr/share/ (when several names match, pressing Tab twice lists them all). With cd /et, one Tab completed it to /etc/. Tab completion saves typing and avoids typos.



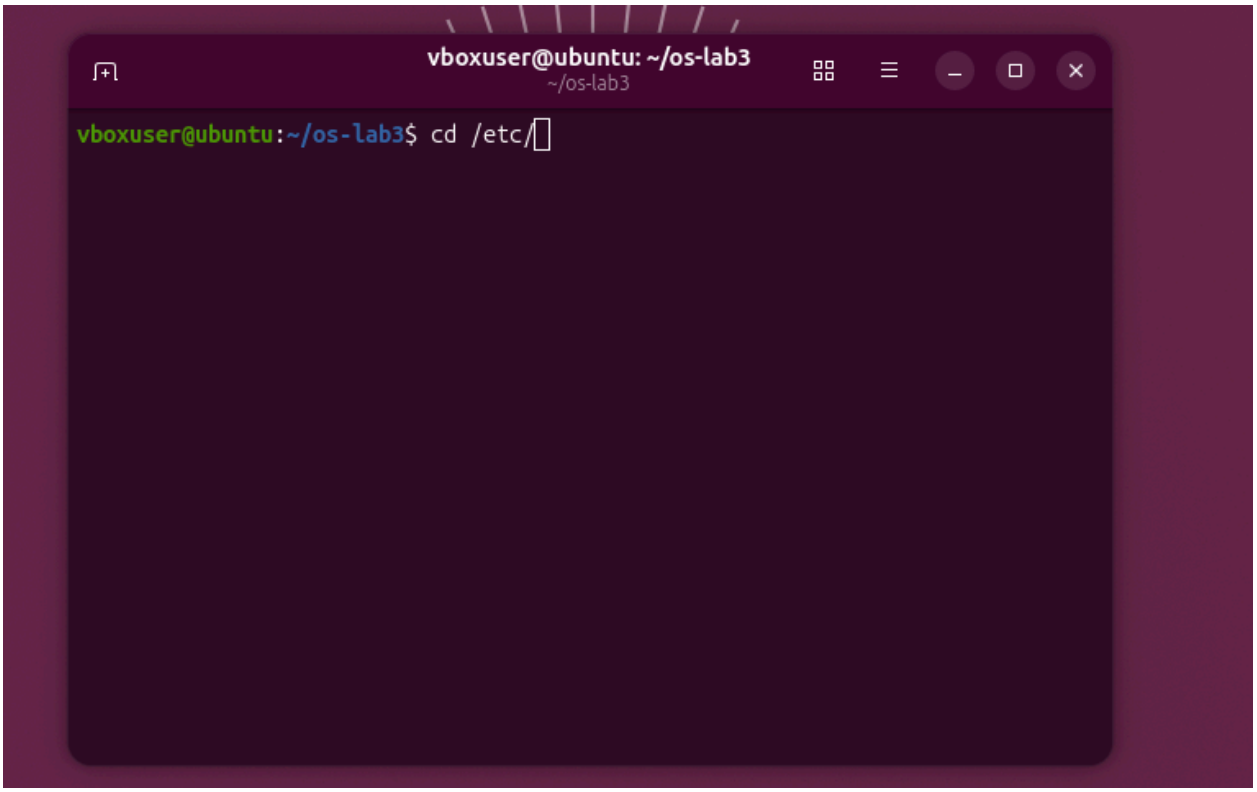


Figure 1.1 – Tab completion

```
date
whoami
pwd
(press Ctrl+R, type "dat", press Enter)
```

Observation: Ctrl+R opens reverse-i-search: as you type, the shell shows the most recent command in the history that contains those letters; Enter runs it (Ctrl+R again goes to an older match). Here the typed letters found the command date and executed it again.

1.2 Built-in or external, and the PATH

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
echo $PATH | tr ':' '\n' | wc -l
nosuchprog ; echo "exit code: $?"
```

Output:

vboxuser@ubuntu:~$ type cd ; type ls ; type echo ; which ls

cd is a shell builtin

ls is aliased to `ls --color=auto'

echo is a shell builtin

/usr/bin/ls

vboxuser@ubuntu:~$ echo $PATH

/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin

vboxuser@ubuntu:~$ echo $PATH | tr ':' '\n'

/usr/local/sbin

/usr/local/bin

/usr/sbin

/usr/bin

/sbin

/bin

/usr/games

/usr/local/games

/snap/bin

/snap/bin

vboxuser@ubuntu:~$ echo $PATH | tr ':' '\n' | wc -l

10

vboxuser@ubuntu:~$ nosuchprog ; echo "exit code: $?"

nosuchprog: command not found

exit code: 127

vboxuser@ubuntu:~$

1.3 Help without a browser

```
man ls            # inside: /sort  then n for next, q to quit
ls --help | head
man -k "list directory"
help cd
```

man shows the full manual page of a program, ls --help prints a short summary, man -k searches manual page descriptions by keyword, and help is the bash help for built-in commands such as cd.

Outputs:

```
vboxuser@ubuntu:~$ man ls
vboxuser@ubuntu:~$ ls --help | head
List directory contents.
Ignore files and directories starting with a '.' by default

Usage: ls [OPTION]... [FILE]...

Arguments:
  [paths]...

Options:
      --help
vboxuser@ubuntu:~$ man -k "list directory"
dir (1)                  - List directory contents. Ignore files and
directories ...
gnudir (1)           - list directory contents
gnuls (1)            - list directory contents
gnuvdir (1)          - list directory contents
ls (1)                   - List directory contents. Ignore files and
directories ...
ntfsls (8)           - list directory contents on an NTFS filesystem
rust-dir (1)             - List directory contents. Ignore files and
directories ...
rust-ls (1)              - List directory contents. Ignore files and
directories ...
rust-vdir (1)           - List directory contents. Ignore files and
directories ...
vdir (1)                 - List directory contents. Ignore files and
directories ...
vboxuser@ubuntu:~$ help cd
cd: cd [-L|[-P [-e]]] [-@] [dir]
```

Change the shell working directory.

Change the current directory to DIR. The default DIR is the value of the

HOME shell variable. If DIR is "-", it is converted to $OLDPWD.

The variable CDPATH defines the search path for the directory containing

DIR. Alternative directory names in CDPATH are separated by a colon (:).

A null directory name is the same as the current directory. If DIR begins

with a slash (/), then CDPATH is not used.

If the directory is not found, and the shell option `cdable_vars' is set,

the word is assumed to be a variable name. If that variable has a value,

its value is used for DIR.

Options:

-L force symbolic links to be followed: resolve symbolic links in DIR after processing instances of `..'

-P use the physical directory structure without following symbolic links: resolve symbolic links in DIR before processing instances of `..'

-e if the -P option is supplied, and the current working directory cannot be determined successfully, exit with a non-zero status

-@ on systems that support it, present a file with extended attributes as a directory containing the file attributes

```
       The default is to follow symbolic links, as if `-L' were
specified.

     `..' is processed by removing the immediately previous pathname
component

    back to a slash or the beginning of DIR.


    Exit Status:

      Returns 0 if the directory is changed, and if $PWD is set
successfully when

    -P is used; non-zero otherwise.

vboxuser@ubuntu:~$
```

Answers to the Observe questions

1. Which of cd, ls, echo are built-ins, and why must cd be one? cd and echo are shell built-ins; ls is an external program stored as a file on disk (found with which ls). cd must be a built-in because every program runs as a separate child process with its own current working directory. If cd were an external program, it would change the directory of its own child process only, and that process would exit immediately, leaving the shell in the old directory. Only the shell itself can change its own working directory.

2. How many directories are in PATH and where is ls? PATH is the list of directories, separated by colons, in which the shell looks for an external command, from left to right.

3. Exit code of a command that is not found: 127. (0 means success, any non-zero value means an error; 127 specifically means "command not found".)

4. Which option of ls sorts by modification time? -t (newest first); combined with -l it shows details (ls -lt), and -r reverses the order (ls -ltr).

Part 2. What the shell does to your line

2.1 Setup

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

Output

```
vboxuser@ubuntu:~$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play

vboxuser@ubuntu:~/os-lab3/play$ touch a.c b.c file1.txt file2.txt
notes.txt "alfa beta.txt"

vboxuser@ubuntu:~/os-lab3/play$ ls

 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

2.2 Predictions and real results

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	print all the .c files to the terminal	a.c b.c	The shell expanded the glob *.c into the matching file names before echo ran.
`echo file?.txt`	matches the question mark with exactly one character	file1.txt file2.txt	? matches exactly one character.
`echo [a-b]*`	files that start with a or b	a.c alfa beta.txt b.c	Any name starting with a or b; the name with a space stays ONE argument because it came from expansion.
`echo *.xyz`	i don't know	*.xyz	No file matches, so bash leaves the pattern unchanged (literal text).
`ls alfa beta.txt`	list files that are called alpha or beta	ls: cannot access 'alfa': No such file or directory ls: cannot access 'beta.txt': No such file or directory	The shell split the line at the space into two arguments, alfa and beta.txt.
`ls "alfa beta.txt"`	keep all the arguments as a single one	alfa beta.txt	Double quotes keep the name as a single argument.

Command	My prediction	Real output	Explanation (if different)
`echo '$HOME' "$HOME" \$HOME`	Literal, expanded, literal	$HOME /home/<user> $HOME	Single quotes: literal. Double quotes: the variable is expanded. Backslash: the next character is literal.
`echo {1..5} ; echo $((7 * 6))`	numbers 1 to 5 and 42	1 2 3 4 5 42	Brace expansion generates the sequence; $(( )) is arithmetic expansion.
`echo "today is $(date +%A), I am $(whoami)"`	will print todays weekday and my username	today is monday, I am vboxuser	$( ) runs the command and substitutes its output into the line.
`touch log{1..3}.txt && ls log*`	i dont know	log1.txt log2.txt log3.txt	Brace expansion created log1.txt log2.txt log3.txt; log* then matched them.

Outputs:

*vboxuser@ubuntu:~/os-lab3/play$ echo *.c*

a.c b.c

vboxuser@ubuntu:~/os-lab3/play$ echo file?.txt

file1.txt file2.txt

*vboxuser@ubuntu:~/os-lab3/play$ echo [a-b]**

a.c alfa beta.txt b.c

*vboxuser@ubuntu:~/os-lab3/play$ echo *.xyz*

**.xyz*

vboxuser@ubuntu:~/os-lab3/play$

vboxuser@ubuntu:~/os-lab3/play$ ls alfa beta.txt

ls: cannot access 'alfa': No such file or directory

ls: cannot access 'beta.txt': No such file or directory

vboxuser@ubuntu:~/os-lab3/play$ ls "alfa beta.txt"

'alfa beta.txt'

vboxuser@ubuntu:~/os-lab3/play$ ls alfa\ beta.txt

'alfa beta.txt'

vboxuser@ubuntu:~/os-l

vboxuser@ubuntu:~/os-lab3/play$ echo 'HOME'"HOME" \$HOME

$HOME /home/vboxuser $HOME

*vboxuser@ubuntu:~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))*

1 2 3 4 5

42

vboxuser@ubuntu:~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"

today is Monday, I am vboxuser

*vboxuser@ubuntu:~/os-lab3/play$ touch log{1..3}.txt && ls log**

log1.txt log2.txt log3.txt

vboxuser@ubuntu:~/os-lab3/play$

Answers to the Observe questions

1. See the table above (prediction, real output, explanation).

2. Who produced the file names for echo *.c? The shell did. Globbing happens before the command is started: bash replaces *.c with the list of matching files, and echo simply receives a.c and b.c as ordinary arguments and prints them. echo itself knows nothing about files.

3. Why did ls alfa beta.txt fail, and two fixes? The shell splits the line into words at spaces, so ls received two arguments, alfa and beta.txt, and neither file exists. Fix 1: quote the name, ls "alfa beta.txt". Fix 2: escape the space with a backslash, ls alfa\ beta.txt (Tab completion also inserts the backslash automatically).

4. Single vs double quotes: inside single quotes everything is literal, so nothing is expanded ($HOME stays $HOME). Inside double quotes spaces and wildcards are protected, but $variables, $( ) command substitution and the backslash are still interpreted ($HOME becomes /home/<user>).

Why does ls show 'alfa beta.txt' in quotes? When writing to a terminal, GNU ls puts quotes around names that contain special characters such as spaces, so that you can copy the name and paste it back as one single argument. The quotes are not part of the real file name (ls | cat prints it without quotes).

Part 3. Redirection, chaining and jobs

3.1 Separate stdout and stderr

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
```

```
vboxuser@ubuntu:~/os-lab3/play$ cd ~/os-lab3

vboxuser@ubuntu:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt

vboxuser@ubuntu:~/os-lab3$ wc -l out.txt ; cat err.txt

238 out.txt

ls: cannot access '/nope': No such file or directory

vboxuser@ubuntu:~/os-lab3$
```

3.2 Order of redirections

```
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
ls /etc /nope 2>&1 > all.txt ; tail -2 all.txt
```

Output:

```
vboxuser@ubuntu:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2
all.txt

xml

zsh_command_not_found

vboxuser@ubuntu:~/os-lab3$ ls /etc /nope 2>&1 > all.txt ; tail -2
all.txt

ls: cannot access '/nope': No such file or directory

xml

zsh_command_not_found

vboxuser@ubuntu:~/os-lab3$
```

3.3 Append, input redirection

```
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
wc -l access.log
```

Output:

vboxuser@ubuntu:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md

first

second

vboxuser@ubuntu:~/os-lab3$ wc -l < access.log

500

vboxuser@ubuntu:~/os-lab3$ wc -l access.log

500 access.log

vboxuser@ubuntu:~/os-lab3$

3.4 Operators ; && ||

```
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
false ; echo "A: runs after ;"
false && echo "B: runs after &&"
false || echo "C: runs after ||"
```

Output

vboxuser@ubuntu:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"

failed with code 2

vboxuser@ubuntu:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3

I am in /home/vboxuser/os-lab3/tmp

vboxuser@ubuntu:~/os-lab3$ false ; echo "A: runs after ;"

A: runs after ;

vboxuser@ubuntu:~/os-lab3$ false && echo "B: runs after &&"

vboxuser@ubuntu:~/os-lab3$ false || echo "C: runs after ||"

C: runs after ||

vboxuser@ubuntu:~/os-lab3$

3.5 Background jobs

```
sleep 100 &
jobs ; kill %1 ; jobs
```

Output

```
vboxuser@ubuntu:~/os-lab3$ sleep 100 &

[1] 6199

vboxuser@ubuntu:~/os-lab3$ jobs ; kill %1 ; jobs

[1]+  Running                 sleep 100 &

[1]+  Running                 sleep 100 &

[1]+  Terminated              sleep 100

vboxuser@ubuntu:~/os-lab3$
```

The trailing & starts the command in the background, so the prompt returns immediately. jobs lists background jobs ([1] Running ...), kill %1 terminates job number 1, and the second jobs shows it as Terminated.

Answers to the Observe questions

1. Which file got the error and which the listing? The listing of /etc (standard output, stream 1) went to out.txt; the error message "ls: cannot access '/nope': No such file or directory" (standard error, stream 2) went to err.txt. That is why wc -l out.txt counted the listing lines and cat err.txt showed only the error.

2. Difference between > all.txt 2>&1 and 2>&1 > all.txt: the shell processes redirections from left to right. In "> all.txt 2>&1", stdout is first sent to all.txt, then stderr is made a copy of wherever stdout points now (the file), so BOTH the listing and the error end up in all.txt. In "2>&1 > all.txt", stderr is first made a copy of stdout while stdout still points to the terminal, and only afterwards stdout is sent to the file; so the error message appears on the screen and only the listing is in all.txt.

3. Why does wc -l < access.log print only a number? With "<" the shell opens the file and gives it to wc as standard input; wc just sees a stream of data and never learns a file name, so it prints only the count. With wc -l access.log, wc opens the file itself from its argument, knows the name and prints it after the number.

4. Operators:

- ; runs the commands one after another, regardless of whether the first one succeeded.
- && runs the second command only if the first one succeeded (exit code 0).
- || runs the second command only if the first one failed (non-zero exit code).

Part 4. Answering questions with one-liners

Each line of access.log has five fields separated by spaces: date, time, user, result (OK or FAIL), page. The data set was generated with the script from the lab description and has 500 lines (check: wc -l access.log prints 500 access.log). All commands below were saved in oneliners.sh, one per question, and are run inside ~/os-lab3.

Q1. How many requests failed?

```
grep -c FAIL access.log
```

```
100
```

grep -c counts the lines that contain the word FAIL.

Output

```
vboxuser@ubuntu:~/os-lab3$ grep -c FAIL access.log
100
```

Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u | wc -l ; cut -d' ' -f5 access.log | sort
-u
```

```
4
/page0
/page1
/page2
/page3
```

cut -d' ' -f5 takes the 5th space-separated field (the page); sort -u sorts and removes duplicates; wc -l counts what is left.

Output

```
vboxuser@ubuntu:~/os-lab3$ cut -d' ' -f5 access.log | sort -u | wc -l
; cut -d' ' -f5 access.log | sort -u
4
/page0
/page1
/page2
/page3
```

Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c
```

```
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

uniq only collapses adjacent duplicates, which is why sort comes first; -c prefixes each line with its count. Check: 4 x 125 = 500, as required.

Q4. Which user or users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk
'NR==1{m=$1} $1==m'
```

```
     15 user5
     15 user3
```

The pipeline counts failures per user and sorts them in descending order; the final awk keeps every line whose count equals the maximum (so that ties are shown). The full per-user counts are user0: 14, user1: 14, user2: 14, user3: 15, user4: 14, user5: 15, user6: 14, which add up to 100 = Q1.

Q5. How many requests did user3 make, and how many of them failed?

```
grep -c 'user3 ' access.log ; grep 'user3 ' access.log | grep -c FAIL
```

```
72
15
```

user3 made 72 requests, 15 of which failed. The space after user3 in the pattern avoids matching other names that begin with user3.

Q6. Print the last 3 failed requests, showing only time and user.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

```
10:10 user0
10:15 user5
10:20 user3
```

tail -3 keeps the last three failed lines; cut -f2,3 keeps only the time and user fields.

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

The 7th colon-separated field of each /etc/passwd line is the login shell. Accounts with /usr/sbin/nologin (or /bin/false) are system accounts that cannot log in interactively.

Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?

```
ls /etc | wc -l
ls -l /etc | wc -l
ls -l /etc | head -3
```

Explanation: the long format (-l) prints one line per entry like the short format, but it also prints one extra header line at the top, "total N", which shows the number of disk blocks used by the listed files. wc -l counts that header as a line, so the second number is exactly one larger than the first.

Full run of oneliners.sh

```
cd ~/os-lab3
bash oneliners.sh
```

Output:

```
vboxuser@ubuntu:~/os-lab3$ bash oneliners.sh
100
4
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
```

```
    125 /page2
    125 /page3
     15 user5
     15 user3
72
15
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
50
```

Bonus. Failed requests whose minute ends in 0

```
grep -E ' 10:[0-5]0 .*FAIL' access.log | wc -l
```

Result: 50. The extended regular expression 10:[0-5]0 matches 10:00, 10:10, ... 10:50 (the minute ends in 0), and .*FAIL requires the word FAIL later on the same line.

Part 5. Your first script

report.sh was created in ~/os-lab3 from the template in the lab sheet and the TODO was completed with a for loop over the users user0 ... user6:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
    x=$(grep -c "user$n FAIL" "$log")
    echo "user$n: $x failures"
done
```

```
cd ~/os-lab3
nano report.sh          # paste the script above, save
cat report.sh
./report.sh access.log          # (try BEFORE chmod: Permission denied)
chmod +x report.sh
./report.sh access.log
./report.sh nope.log ; echo "exit code: $?"
```

Output

vboxuser@ubuntu:~/os-lab3$ touch report.sh

vboxuser@ubuntu:~/os-lab3$ cd ~/os-lab3

vboxuser@ubuntu:~/os-lab3$ nano report.sh

vboxuser@ubuntu:~/os-lab3$ cat report.sh

#!/bin/bash

usage: ./report.sh access.log

log=$1

if [! -f "$log"]; then

  echo "no such file: $log" >&2

  exit 1

fi

total=($wc - l <$ "log")

fails=($grep - cFAIL$"log")

echo "$total requests, $fails failed"

for n in 0 1 2 3 4 5 6; do

```
  x=($grep -c "user$n FAIL" "$log")
  echo "user$n: $x failures"
done
```

```
vboxuser@ubuntu:~/os-lab3$ ./report.sh access.log
bash: ./report.sh: Permission denied
vboxuser@ubuntu:~/os-lab3$ chmod +x report.sh
vboxuser@ubuntu:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
vboxuser@ubuntu:~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
vboxuser@ubuntu:~/os-lab3$
```

Answers to the Observe questions

1. What does #!/bin/bash do, and what if you run ./report.sh before chmod +x? The first line ("shebang") tells the kernel which interpreter must run the file, here /bin/bash; without it the system would not know how to execute the text file. Before chmod +x the file has no execute permission, so the shell refuses to run it: "bash: ./report.sh: Permission denied".

2. Why ./report.sh and not just report.sh? Without a slash the shell searches for the command only in the directories listed in PATH, and the current directory is deliberately not in PATH (for security: a malicious file named like a standard command could otherwise be run by accident). ./ gives an explicit path to the current directory, so no PATH lookup is done.

3. Why >&2 and exit 1? >&2 sends the message to standard error instead of standard output, so error messages do not get mixed into the normal output when it is redirected or piped (the user still sees them on the screen). exit 1 ends the script with a non-zero exit code, which tells the caller that the script failed; that is what $?, &&, || and cron rely on. Exit code 0 (the default at the end) means success.

Bonus: running the script with cron

```
crontab -e
# add this line (replace USER with your username):
* * * * * /home/USER/os-lab3/report.sh /home/USER/os-lab3/access.log >>
/home/USER/os-lab3/report.log 2>&1
# wait 2 minutes, then:
crontab -l
cat ~/os-lab3/report.log
# finally remove the line:
crontab -e     # (or crontab -r to delete the whole crontab)
```

Conclusions

In this laboratory work I worked with the Linux bash shell from the command line. I used Tab completion, Ctrl+R history search and the built-in help (man, --help, help) to work faster, and I learned that commands are either shell built-ins (cd, echo) or external programs found through the directories of PATH; a command that is not found returns exit code 127.

I saw that the shell, not the command, performs globbing, brace expansion, variable and command substitution and quote handling before the command starts, and that single quotes, double quotes and backslash control this expansion. Predicting the result first made the differences between my expectations and the real behavior clear.

I redirected stdout and stderr to files, understood why the order of redirections matters (> file 2>&1 versus 2>&1 > file), chained commands with ;, && and ||, and ran and stopped a background job. Finally, I answered eight questions about a 500-line log with one-line pipelines of grep, cut, sort, uniq, wc, head and tail, checked the results against each other (Q3 adds up to 500, the per-user failures add up to the 100 of Q1), and turned them into the script report.sh with a loop, an argument check, an error message on stderr and a meaningful exit code.